EdgeShield: Partner B Work Package

DNS engine, benchmarking, bottleneck simulation, packet analysis and report section. Everything you still need to do, in order, with a paste-ready prompt for Antigravity.

1. Where things stand

Item	Status	Notes
DNS lookup (local name) via dig	Done (earlier session)	webserver.lan -> 192.168.1.50, NOERROR, 0 ms
HTTP proxy cache miss and hit (Partner A)	Done (earlier session)	Miss, then 304 Not Modified, served from disk
Upgraded DNS resolver (sinkhole, upstream, TTL cache)	Written and tested	In section 5. Tested here against a fake upstream; not yet run on your Mac
benchmark.py and plot_results.py	Written and tested	DNS part tested. HTTP proxy mode still needs Partner A's real proxy
15 Mbps throttled test rig	TO DO (you)	Section 3, step 5
Wireshark captures	TO DO (you)	Section 3, step 7
Real measured numbers and figures	TO DO (you)	Only quote numbers you measured yourself
Report section and viva preparation	TO DO (you + Antigravity)	Sections 3 (steps 9-10) and 6

Your role

You are **Partner B**: the UDP side. You own `dns_resolver.py`, `benchmark.py`, the bottleneck simulation, the packet captures, the delay plots, and the benchmarking half of the report. Partner A owns `edgeshield_proxy.py` (TCP, port 8888). Do not edit Partner A's file.

The project in one paragraph

Institutional LANs run at 100 Mbps but reach the Internet through a shared access link of about 15 Mbps. Traffic intensity is I = La/R. As I approaches 1, queuing delay grows without bound, and above 1 the buffer overflows and packets drop. EdgeShield sits at the edge with a TCP caching proxy (Conditional GET, If-Modified-Since, 304) and a UDP DNS resolver (local zone, sinkhole, upstream forwarding, TTL cache), so repeated requests never cross the bottleneck.

2. Read this before you present (honesty checklist)

- **Never quote numbers you did not measure.** The table in the earlier PDF (85.4 ms vs 0.8 ms, 120 MB vs 1.2 MB, 400,106 ms) was illustrative or an anomaly. Use only your CSV results.
- **Do not claim DNS-to-proxy pinning.** In the prototype the DNS resolver and the proxy run side by side and are not wired together. Present them as two coordinated edge services, not a coupled system.
- **The 400,106 ms first-fetch figure** came from a request that left the connection open so the proxy waited for the server to close it. Always send `Connection: close` (benchmark.py does). Do not present that number as a latency result.
- **Figure 3 is a computed model** (L/R and propagation from the link settings), not a measurement. Label it that way.
- **Be able to explain every line** of dns_resolver.py. You will be asked why UDP, what is in the 12-byte header, and how the sinkhole and cache work (section 6 has answers).

3. Your tasks, in order, with acceptance criteria

Do these on your Mac in the project folder `~/Desktop/edgeshield_project`. Antigravity can run the commands and edit files for you; the prompt in section 4 tells it how.

#	What to do	Done when
1	Put these files in the project folder: dns_resolver.py, benchmark.py, plot_results.py (section 5) and Partner A's final edgeshield_proxy.py. Run `pip install matplotlib`.	`ls` shows all four files. `python3 -c 'import matplotlib'` prints nothing.
2	Read Partner A's proxy and check it matches the benchmark: listens on 127.0.0.1:8888, accepts an absolute-URI GET (`GET http://example.com/ HTTP/1.1`), stores files in ./edge_storage, and sends If-Modified-Since on a hit. Change only benchmark.py arguments if not.	A written list of any mismatches, and benchmark.py works with the real proxy (one manual curl succeeds).
3	Test the DNS resolver. Terminal 1: `python3 dns_resolver.py`. Terminal 2: `dig @127.0.0.1 -p 5300 webserver.lan A`, then doubleclick.net, then google.com twice.	webserver.lan returns 192.168.1.50. doubleclick.net returns 0.0.0.0. google.com is logged `[DNS UPSTREAM]` the first time and `[DNS CACHE]` the second. dns_log.csv exists with one row per query.
4	Baseline benchmark, no throttling. Start the DNS resolver and Partner A's proxy, then run `python3 benchmark.py --url http://example.com/ --n 50`. Save the terminal output.	results_http.csv has 100 data rows (50 direct, 50 proxy), none with status ERR. results_dns.csv has 40 rows. Move both into a folder named run_baseline.
5	Throttle the link to 15 Mbps down, 2 Mbps up, 50 ms RTT (see section 7). Verify with `ping example.com` that latency rose by about 50 ms.	ping shows the added delay. `ping 127.0.0.1` is still under 1 ms (the LAN side must stay fast).
6	Repeat step 4 with throttling on. Run it in a new folder so CSVs are not overwritten.	A run_throttled folder with its own two CSVs. Direct mean latency is clearly higher than proxy-cached mean latency. If it is not, report that honestly and investigate.
7	Capture packets in Wireshark (section 8): one capture on lo0 for DNS and client-to-proxy, one on en0 for proxy-to-origin traffic. Take the five screenshots listed there.	Five annotated screenshots saved: DNS header flags, sinkhole answer, 200 OK on miss, 304 on revalidation, and the Conversations byte totals.
8	Run `python3 plot_results.py --link-mbps 15 --lan-mbps 100 --rtt-ms 50` inside run_throttled.	fig1_latency.png, fig2_dns_latency.png and fig3_delay_model.png exist and match your CSVs.
9	Fill the results tables in section 6 from your CSVs, then write your report section (structure in section 6).	Every number in the report appears in a CSV or a screenshot. No placeholders left.
10	Rehearse the viva answers in section 6 out loud, once with Partner A.	You can explain header flags, UDP vs TCP, TTL caching, sinkholing and the L/R math without notes.
11	Final check: files named correctly, figures embedded, both partners' names and roles stated.	Submission folder contains report, code, CSVs, figures and screenshots.

Due date for submission: _____ Presentation slot: _____

4. Paste-ready prompt for Antigravity

Open the project folder in Antigravity, attach this PDF (or paste the prompt), and send it. Antigravity runs in your environment, so it can execute the commands and read Partner A's real files.

```
You are helping me finish Partner B's part of a Computer Networks case study called EdgeShield.
Project folder: ~/Desktop/edgeshield_project (macOS). Read every file in it first.

CONTEXT
- Partner A owns edgeshield_proxy.py: a TCP caching proxy on 127.0.0.1:8888 using
  If-Modified-Since / 304. DO NOT EDIT THIS FILE.
- I am Partner B. I own dns_resolver.py (UDP DNS on port 5300: local zone, sinkhole,
  upstream forwarding to 1.1.1.1, TTL cache), benchmark.py, plot_results.py, the
  throttled-link experiment, Wireshark evidence, figures and my report section.
- The three scripts are in the attached PDF, section 5. Create them if missing.

RULES
1. Never invent or estimate measurements. Every number must come from a command you ran.
2. Run commands and show me the real output. If something fails, show the error and stop.
3. Always send "Connection: close" on test requests (the proxy otherwise hangs).
4. Do not claim DNS-to-proxy pinning; the two services are independent.
5. Ask before running any command that needs sudo or changes network settings.

TASKS (do them in order, confirm each acceptance check before moving on)
1. Check Partner A's proxy matches benchmark.py (port, absolute-URI GET, edge_storage,
   If-Modified-Since). Report mismatches; fix only benchmark.py arguments.
2. Start dns_resolver.py. Run dig for webserver.lan, doubleclick.net, google.com (twice).
   Expected: 192.168.1.50 / 0.0.0.0 / UPSTREAM then CACHE in the log.
3. Start the proxy and run: python3 benchmark.py --url http://example.com/ --n 50
   in a folder named run_baseline. Confirm 100 HTTP rows and 40 DNS rows, no ERR.
4. Walk me through enabling a 15 Mbps down / 2 Mbps up / 50 ms RTT throttle
   (Network Link Conditioner preferred; dnctl+pfctl as fallback). Verify with ping.
5. Repeat the benchmark with throttling in run_throttled, then run
   python3 plot_results.py --link-mbps 15 --lan-mbps 100 --rtt-ms 50
6. Summarise results as tables (mean, median, p95, min, max per mode; DNS per name;
   speedup) computed from the CSVs, not from memory.
7. Draft my report section (Partner B) using only those results, plus a 2-minute
   spoken explanation I can use when presenting.
8. List anything in the results that looks odd (cached slower than direct, errors,
   timeouts) instead of hiding it.

When finished, give me a checklist of every file produced and what each one proves.
```

5. Code (copy these into your project folder)

All three files were run on a test machine. The DNS resolver was verified for all four paths (local, sinkhole, upstream, cache) against a stand-in upstream server, because the test machine has no Internet access. It has **not** been run on a Mac against the real 1.1.1.1 or against Partner A's real proxy, so run section 3 steps 3 and 4 before trusting it.

What changed from the earlier version of dns_resolver.py

- The upstream cache was never filled, so the cache branch could not trigger. Upstream answers are now stored with the smallest TTL found in the answer records and reused until they expire.
- Cached replies keep the client's own transaction ID (the first two bytes), as DNS requires.
- Domain names are lower-cased, and only A queries are answered locally. Other query types are forwarded (or get an empty NOERROR if sinkholed), instead of returning a wrong A record.
- Malformed packets no longer crash the server, and every query is logged to dns_log.csv with its path and latency.

dns_resolver.py

```
"""EdgeShield DNS engine (Partner B).

UDP DNS resolver with four resolution paths, checked in this order:
  1. SINKHOLE  - blocked domains get 0.0.0.0 (A) or an empty NOERROR (other types)
  2. LOCAL     - static zone records (*.lan) answered from memory
  3. CACHE     - upstream answers cached until their TTL expires
  4. UPSTREAM  - unknown names forwarded to a public resolver, answer cached

Run:      python3 dns_resolver.py
Options:  EDGESHIELD_DNS_PORT (default 5300), EDGESHIELD_UPSTREAM (default 1.1.1.1:53)
Logs:     dns_log.csv  (timestamp, client, qname, qtype, path, latency_ms)
"""
import csv
import os
import struct
import time
from socket import AF_INET, SOCK_DGRAM, socket

DNS_PORT = int(os.environ.get("EDGESHIELD_DNS_PORT", "5300"))
_up_host, _, _up_port = os.environ.get("EDGESHIELD_UPSTREAM", "1.1.1.1:53").partition(":")
UPSTREAM_DNS = (_up_host, int(_up_port or 53))
UPSTREAM_TIMEOUT = 2.0
LOG_FILE = "dns_log.csv"

LOCAL_ZONES = {
    "webserver.lan": "192.168.1.50",
    "gateway.lan": "192.168.1.1",
    "proxy.lan": "127.0.0.1",
}
SINKHOLE_LIST = {
    "adserver.tracker.com",
    "telemetry.analytics.net",
    "doubleclick.net",
}

QTYPE_A = 1
# cache key: (qname, qtype) -> (raw_response_bytes, expiry_epoch)
dns_cache = {}
stats = {"SINKHOLE": 0, "LOCAL": 0, "CACHE": 0, "UPSTREAM": 0, "FAIL": 0}


def parse_question(buf):
    """Return (qname_lowercase, qtype, question_bytes) from a query packet."""
    idx, labels = 12, []
    while idx < len(buf) and buf[idx] != 0:
        n = buf[idx]
        labels.append(buf[idx + 1: idx + 1 + n].decode("ascii", "ignore"))
        idx += 1 + n
    qtype = struct.unpack("!H", buf[idx + 1: idx + 3])[0]
    end = idx + 5  # zero byte + QTYPE(2) + QCLASS(2)
    return ".".join(labels).lower(), qtype, buf[12:end]
```

```
def build_response(tx_id, question, ip=None, rcode=0, ttl=60):
    """Build a response: one A record if ip is given, else an empty answer."""
    flags = 0x8180 | rcode  # QR=1, RD=1, RA=1
    header = tx_id + struct.pack("!HHHHH", flags, 1, 1 if ip else 0, 0, 0)
    if not ip:
        return header + question
    rr = b"\xc0\x0c" + struct.pack("!HHIH", 1, 1, ttl, 4)
    return header + question + rr + bytes(int(p) for p in ip.split("."))


def _skip_name(buf, off):
    while True:
        n = buf[off]
        if n == 0:
            return off + 1
        if n & 0xC0 == 0xC0:  # compression pointer
            return off + 2
        off += 1 + n


def min_answer_ttl(resp):
    """Smallest TTL among answer records (None if no answers)."""
    try:
        qd, an = struct.unpack("!HH", resp[4:8])
        off = 12
        for _ in range(qd):
            off = _skip_name(resp, off) + 4
        ttls = []
        for _ in range(an):
            off = _skip_name(resp, off)
            _, _, ttl, rdlen = struct.unpack("!HHIH", resp[off: off + 10])
            ttls.append(ttl)
            off += 10 + rdlen
        return min(ttls) if ttls else None
    except Exception:
        return None


def forward_upstream(data):
    s = socket(AF_INET, SOCK_DGRAM)
    s.settimeout(UPSTREAM_TIMEOUT)
    try:
        s.sendto(data, UPSTREAM_DNS)
        resp, _ = s.recvfrom(4096)
        return resp
    except OSError:
        return None
    finally:
        s.close()


def resolve(data):
    """Return (response_bytes, path_label) for one query packet."""
    tx_id = data[:2]
    qname, qtype, question = parse_question(data)

    if qname in SINKHOLE_LIST:
        stats["SINKHOLE"] += 1
        ip = "0.0.0.0" if qtype == QTYPE_A else None
        return build_response(tx_id, question, ip, ttl=300), "SINKHOLE"

    if qname in LOCAL_ZONES and qtype == QTYPE_A:
        stats["LOCAL"] += 1
        return build_response(tx_id, question, LOCAL_ZONES[qname]), "LOCAL"

    key, now = (qname, qtype), time.time()
    hit = dns_cache.get(key)
    if hit and hit[1] > now:
        stats["CACHE"] += 1
        return tx_id + hit[0][2:], "CACHE"  # reuse answer, keep client's tx id

    resp = forward_upstream(data)
    if resp is None:
        stats["FAIL"] += 1
        return build_response(tx_id, question, rcode=2), "FAIL"  # SERVFAIL
```

```
    stats["UPSTREAM"] += 1
    ttl = min_answer_ttl(resp)
    if ttl:
        dns_cache[key] = (resp, now + ttl)
    return resp, "UPSTREAM"


def main():
    sock = socket(AF_INET, SOCK_DGRAM)
    sock.bind(("0.0.0.0", DNS_PORT))
    new_log = not os.path.exists(LOG_FILE)
    log_f = open(LOG_FILE, "a", newline="")
    log = csv.writer(log_f)
    if new_log:
        log.writerow(["timestamp", "client", "qname", "qtype", "path", "latency_ms"])
    print(f"[*] EdgeShield DNS on UDP {DNS_PORT}, upstream {UPSTREAM_DNS[0]}:{UPSTREAM_DNS[1]}")
    try:
        while True:
            data, addr = sock.recvfrom(4096)
            if len(data) < 17:
                continue
            t0 = time.perf_counter()
            try:
                resp, path = resolve(data)
            except Exception as exc:  # malformed packet: never crash the server
                print(f"[DNS ERROR] {exc}")
                continue
            sock.sendto(resp, addr)
            ms = (time.perf_counter() - t0) * 1000
            qname, qtype, _ = parse_question(data)
            print(f"[DNS {path}] {qname} type={qtype} {ms:.2f} ms")
            log.writerow([time.time(), addr[0], qname, qtype, path, f"{ms:.3f}"])
            log_f.flush()
    except KeyboardInterrupt:
        print(f"\n[*] Stopped. Stats: {stats}")
    finally:
        log_f.close()
        sock.close()


if __name__ == "__main__":
    main()
```

benchmark.py

```
"""EdgeShield benchmark harness (Partner B).

Measures (1) HTTP latency direct vs through the caching proxy and (2) DNS
latency per resolution path. Writes results_http.csv and results_dns.csv.

Usage (start dns_resolver.py and Partner A's proxy first):
    python3 benchmark.py --url http://example.com/ --n 50
    python3 benchmark.py --skip-http          # DNS only
    python3 benchmark.py --skip-dns           # HTTP only
"""
import argparse
import csv
import random
import statistics
import struct
import time
from socket import AF_INET, SOCK_DGRAM, SOCK_STREAM, socket
from urllib.parse import urlparse


def http_get(url, proxy=None, timeout=60):
    """One GET. proxy=(host, port) sends an absolute-URI request via the proxy."""
    u = urlparse(url)
    host, port = u.hostname, u.port or 80
    path = u.path or "/"
    if u.query:
        path += "?" + u.query
    target = proxy if proxy else (host, port)
    line = f"GET {url} HTTP/1.1" if proxy else f"GET {path} HTTP/1.1"
    req = f"{line}\r\nHost: {host}\r\nUser-Agent: edgeshield-bench\r\nConnection: close\r\n\r\n"
```

```
    t0 = time.perf_counter()
    s = socket(AF_INET, SOCK_STREAM)
    s.settimeout(timeout)
    try:
        s.connect(target)
        s.sendall(req.encode())
        buf = b""
        while True:
            chunk = s.recv(65536)
            if not chunk:
                break
            buf += chunk
    finally:
        s.close()
    ms = (time.perf_counter() - t0) * 1000
    first = buf.split(b"\r\n", 1)[0].decode("latin1", "ignore")
    status = first.split(" ")[1] if first.startswith("HTTP/") and " " in first else "ERR"
    return ms, len(buf), status


def dns_query(name, server, qtype=1, timeout=3):
    """Send one DNS query, return (ms, rcode, answer_count, first_ipv4_or_empty)."""
    qid = random.randint(0, 65535)
    q = struct.pack("!HHHHHH", qid, 0x0100, 1, 0, 0, 0)
    for label in name.split("."):
        q += bytes([len(label)]) + label.encode()
    q += b"\x00" + struct.pack("!HH", qtype, 1)
    s = socket(AF_INET, SOCK_DGRAM)
    s.settimeout(timeout)
    t0 = time.perf_counter()
    try:
        s.sendto(q, server)
        resp, _ = s.recvfrom(4096)
    except OSError:
        return (time.perf_counter() - t0) * 1000, -1, 0, ""
    finally:
        s.close()
    ms = (time.perf_counter() - t0) * 1000
    rcode = struct.unpack("!H", resp[2:4])[0] & 0x000F
    an = struct.unpack("!H", resp[6:8])[0]
    ip = ".".join(str(b) for b in resp[-4:]) if an and qtype == 1 else ""
    return ms, rcode, an, ip


def summary(label, vals):
    vals = sorted(vals)
    p95 = vals[max(0, int(len(vals) * 0.95) - 1)]
    print(f"  {label:<22} mean={statistics.mean(vals):9.2f}  median={statistics.median(vals):9.2f}"
          f"  p95={p95:9.2f}  min={vals[0]:9.2f}  max={vals[-1]:9.2f} ms")


def bench_http(args):
    rows = []
    px = args.proxy.split(":")
    proxy = (px[0], int(px[1]))
    for mode, p in (("direct", None), ("proxy", proxy)):
        print(f"\n[HTTP] mode={mode}  n={args.n}  url={args.url}")
        for i in range(1, args.n + 1):
            try:
                ms, size, status = http_get(args.url, p)
            except OSError as exc:
                print(f"  req {i:03d}: ERROR {exc}")
                rows.append([mode, i, "", 0, "ERR"])
                continue
            rows.append([mode, i, f"{ms:.3f}", size, status])
            print(f"  req {i:03d}: {ms:9.2f} ms  {size:8d} B  status={status}")
    with open("results_http.csv", "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["mode", "request", "latency_ms", "bytes", "status"])
        w.writerows(rows)
    print("\n---- HTTP summary ----")
    for mode in ("direct", "proxy"):
        ok = [float(r[2]) for r in rows if r[0] == mode and r[2] != ""]
        if ok:
            summary(mode + " (all)", ok)
```

```
        if mode == "proxy" and len(ok) > 1:
            summary("proxy (cached, 2..n)", ok[1:])
    d = [float(r[2]) for r in rows if r[0] == "direct" and r[2] != ""]
    p = [float(r[2]) for r in rows if r[0] == "proxy" and r[2] != ""]
    if d and len(p) > 1:
        print(f"  speedup (mean direct / mean proxy-cached): "
              f"{statistics.mean(d) / statistics.mean(p[1:]):.2f}x")


def bench_dns(args):
    dh = args.dns.split(":")
    server = (dh[0], int(dh[1]))
    names = ["webserver.lan", "doubleclick.net", "example.com", "example.org"]
    rows = []
    print(f"\n[DNS] server={args.dns}  {args.dns_repeats} queries per name")
    for name in names:
        for i in range(1, args.dns_repeats + 1):
            ms, rcode, an, ip = dns_query(name, server)
            rows.append([name, i, f"{ms:.3f}", rcode, an, ip])
            print(f"  {name:<18} #{i:02d} {ms:8.2f} ms rcode={rcode} answers={an} {ip}")
    with open("results_dns.csv", "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["qname", "query", "latency_ms", "rcode", "answers", "ip"])
        w.writerows(rows)
    print("\n---- DNS summary ----")
    for name in names:
        v = [float(r[2]) for r in rows if r[0] == name and r[3] == 0]
        if v:
            summary(name, v)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--url", default="http://example.com/")
    ap.add_argument("--n", type=int, default=50)
    ap.add_argument("--proxy", default="127.0.0.1:8888")
    ap.add_argument("--dns", default="127.0.0.1:5300")
    ap.add_argument("--dns-repeats", type=int, default=10)
    ap.add_argument("--skip-http", action="store_true")
    ap.add_argument("--skip-dns", action="store_true")
    args = ap.parse_args()
    if not args.skip_dns:
        bench_dns(args)
    if not args.skip_http:
        bench_http(args)


if __name__ == "__main__":
    main()
```

plot_results.py

```
"""Generate report figures from results_http.csv and results_dns.csv.

    python3 plot_results.py --link-mbps 15 --lan-mbps 100 --rtt-ms 50
Outputs fig1_latency.png, fig2_dns_latency.png, fig3_delay_model.png
"""
import argparse
import csv

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt


def load(path):
    with open(path) as f:
        return list(csv.DictReader(f))


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--link-mbps", type=float, default=15.0)
    ap.add_argument("--lan-mbps", type=float, default=100.0)
    ap.add_argument("--rtt-ms", type=float, default=50.0)
```

```
    a = ap.parse_args()

    http = [r for r in load("results_http.csv") if r["latency_ms"]]
    fig, ax = plt.subplots(figsize=(7, 4))
    for mode, color in (("direct", "#c0392b"), ("proxy", "#2471a3")):
        pts = [(int(r["request"]), float(r["latency_ms"])) for r in http if r["mode"] == mode]
        ax.plot([p[0] for p in pts], [p[1] for p in pts], marker="o", ms=3, label=mode, color=color)
    ax.set_xlabel("Request number")
    ax.set_ylabel("Response time (ms)")
    ax.set_title("Figure 1: HTTP latency, direct vs EdgeShield")
    ax.legend()
    ax.grid(alpha=0.3)
    fig.tight_layout()
    fig.savefig("fig1_latency.png", dpi=200)

    dns = [r for r in load("results_dns.csv") if r["rcode"] == "0"]
    names = sorted({r["qname"] for r in dns})
    means = [sum(float(r["latency_ms"]) for r in dns if r["qname"] == n) /
             max(1, sum(1 for r in dns if r["qname"] == n)) for n in names]
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.bar(names, means, color="#2471a3")
    ax.set_ylabel("Mean DNS latency (ms)")
    ax.set_title("Figure 2: DNS latency by name (local / sinkhole / upstream+cache)")
    plt.setp(ax.get_xticklabels(), rotation=15)
    fig.tight_layout()
    fig.savefig("fig2_dns_latency.png", dpi=200)

    size = max(int(r["bytes"]) for r in http if r["mode"] == "direct")
    bits = size * 8
    parts = {
        "Direct (throttled link)": (bits / (a.link_mbps * 1e6) * 1000, a.rtt_ms),
        "EdgeShield cache hit (LAN)": (bits / (a.lan_mbps * 1e6) * 1000, 0.0),
    }
    fig, ax = plt.subplots(figsize=(7, 4))
    labels = list(parts)
    trans = [parts[k][0] for k in labels]
    prop = [parts[k][1] for k in labels]
    ax.bar(labels, trans, label="d_trans = L/R", color="#2471a3")
    ax.bar(labels, prop, bottom=trans, label="d_prop (RTT)", color="#e67e22")
    ax.set_ylabel("Delay (ms)")
    ax.set_title(f"Figure 3: Delay model for a {size} B object (computed, not measured)")
    ax.legend()
    fig.tight_layout()
    fig.savefig("fig3_delay_model.png", dpi=200)
    print("Saved fig1_latency.png, fig2_dns_latency.png, fig3_delay_model.png")


if __name__ == "__main__":
    main()
```

6. Report section, results tables and viva answers

Suggested structure for Partner B's report section

- **6.1 DNS engine design:** UDP datagram socket on port 5300, the 12-byte header (ID, flags, QDCOUNT, ANCOUNT, NSCOUNT, ARCOUNT), question parsing, Type A resource record layout (name pointer, type, class, TTL, RDLENGTH, RDATA).
- **6.2 Resolution paths:** sinkhole, local zone, TTL cache, upstream forwarding, with the order of checks.
- **6.3 Experimental setup:** Mac model, macOS version, link settings (15 Mbps down, 2 Mbps up, 50 ms RTT), object URL and size, number of requests, tools used.
- **6.4 Results:** the three tables below plus Figures 1 to 3 and the five Wireshark screenshots.
- **6.5 Analysis:** link the results to d_nodal = d_proc + d_queue + d_trans + d_prop and to I = La/R.
- **6.6 Limitations:** single client, one object, loopback LAN, DNS and proxy not integrated, HTTP only (no HTTPS), cache correctness depends on origin Last-Modified behaviour.

Table A: HTTP latency (fill from results_http.csv in run_throttled)

Mode	Mean (ms)	Median (ms)	p95 (ms)	Min (ms)	Max (ms)
Direct					
Proxy, first request (miss)					
Proxy, requests 2 to n (hit)					

Table B: DNS latency (fill from results_dns.csv)

Name	Path expected	Mean (ms)	Median (ms)	Max (ms)
webserver.lan	Local zone			
doubleclick.net	Sinkhole			
example.com	Upstream, then cache			
example.org	Upstream, then cache			

Table C: Access-link bytes (fill from Wireshark Conversations on en0)

Scenario	Bytes over access link	Source
50 direct requests		Wireshark capture 1
50 requests via EdgeShield		Wireshark capture 2
Saving		(direct - EdgeShield) / direct

Likely questions and short answers

Question	Answer
Why UDP for DNS?	A query and reply fit in one small datagram, so there is no handshake. One round trip instead of three-way handshake plus request. Reliability is handled by the client retrying.
What is in the 12-byte header?	ID (2 B), flags (2 B), and four 2-byte counts: questions, answers, authority, additional. Our responses use flags 0x8180: QR=1 (response), RD=1, RA=1, RCODE=0.
How does the cache know when to expire?	On an upstream answer we read the TTL of each answer record, take the smallest, and store the reply until now plus that TTL.

Question	Answer
What does the sinkhole do?	For a blocked name it answers 0.0.0.0, so the client never connects to the ad server and no bandwidth is spent crossing the access link.
Why does caching help the bottleneck?	I = La/R. A cache hit sends only a small 304 check (or nothing) over the access link, which lowers La and keeps I well below 1, so queuing delay stays small.
Does this already exist?	Yes: Squid, Varnish, CDNs, Pi-hole, dnsmasq, Unbound. Our contribution is a from-scratch teaching implementation that measures the delay effects with raw sockets.
What are the limits?	HTTP only, one client, small test set, DNS and proxy not integrated, results depend on the emulated link.

7. Throttling the link on macOS

Option 1 (recommended): Network Link Conditioner

- Install it from Apple's *Additional Tools for Xcode* download (it appears under System Settings).
- Create a custom profile: downlink 15 Mbps, uplink 2 Mbps, delay 50 ms (apply to both directions or set 25 ms each way for a 50 ms round trip, and write down which you chose).
- Enable it, then verify: `ping example.com` should show about 50 ms extra, and `ping 127.0.0.1` should stay under 1 ms.
- Turn it off when you finish. Leaving it on slows your whole Mac.

Option 2 (fallback): dnctl and pfctl

```
sudo dnctl pipe 1 config bw 15Mbit/s delay 25
echo "dummynet out on en0 proto tcp from any to any port 80 pipe 1" | sudo pfctl -f -
sudo pfctl -e
# ... run the experiment ...
sudo pfctl -d
sudo dnctl -q flush
```

I could not test the pfctl route (I cannot run it on a Mac). It replaces the active pf ruleset and interface names vary (en0 is common on Wi-Fi Macs), so let Antigravity check your interface with `route get default` first, and use Option 1 if you can.

Do not forget to record

Mac model and macOS version, throttle profile values, URL tested, object size, date and time, and whether other traffic was running during the test.

8. Wireshark capture plan

#	Capture and filter	What to show and annotate
S1	Interface lo0, filter `udp.port == 5300`. Run dig for webserver.lan.	Expand the DNS header. Circle ID, flags 0x8180, counts, and the answer record with its TTL.
S2	Same capture, run dig for doubleclick.net.	Answer address 0.0.0.0 (the sinkhole reply).
S3	Interface en0, filter `http.request or http.response`, first proxy request.	HTTP 200 OK with the full body size. This is the cache miss crossing the access link.
S4	Same capture, second proxy request.	Request carrying If-Modified-Since, reply HTTP 304 Not Modified with no body. Note the frame length.
S5	Statistics > Conversations on en0, 50 direct requests vs 50 proxy requests (two captures).	Total bytes to the origin for each scenario. This feeds Table C.

Save each capture as a .pcapng file and keep the screenshots with your report. If the first request returns a redirect, use a different plain-HTTP URL so the 200 and 304 are visible.